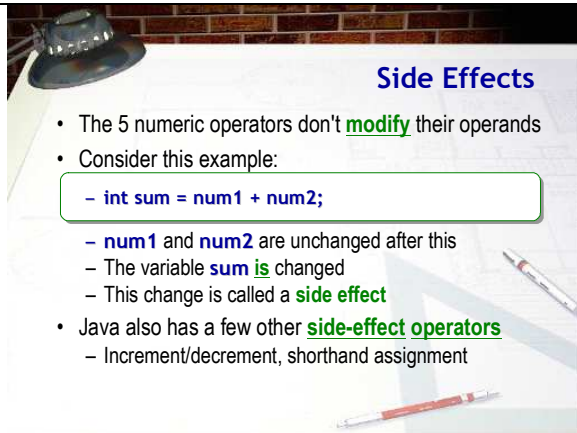




Side Effects

- The 5 numeric operators don't **modify** their operands
- Consider this example:


```
int sum = num1 + num2;
```

 - `num1` and `num2` are unchanged after this
 - The variable `sum` is changed
 - This change is called a **side effect**
- Java also has a few other **side-effect operators**
 - Increment/decrement, shorthand assignment

Slide 1 

Side Effects

Duration: 00:00:53

Advance mode: Auto

Notes:

Hello. Welcome to the CS 170, Java Programming 1 lecture on Side Effects.

The five numeric operators we looked at in the last lesson all have one thing in common: none of them modify their operands. Let me show you what I mean.

When you use the addition operator to add two numbers together, like this:

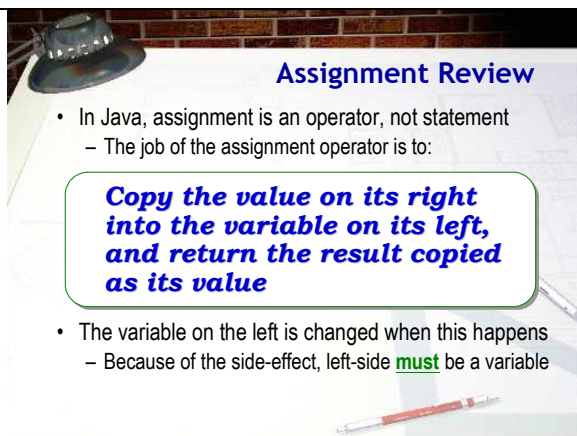
```
int sum = num1 + num2;
```

the operands, `num1` and `num2`, are not changed at all; the variable `sum`, which stores the results of the calculation, is changed, however. This kind of change is called a side effect.

Side effects occur whenever an expression changes the value of one of its operands.

The assignment operator produces a side effect, as do the increment and decrement operators, and the various shorthand assignment operators.

Let's start by first reviewing the assignment operator.



Assignment Review

- In Java, assignment is an operator, not statement
 - The job of the assignment operator is to:

Copy the value on its right into the variable on its left, and return the result copied as its value
- The variable on the left is changed when this happens
 - Because of the side-effect, left-side **must** be a variable

Slide 2 

Assignment Review

Duration: 00:00:45

Advance mode: Auto

Notes:

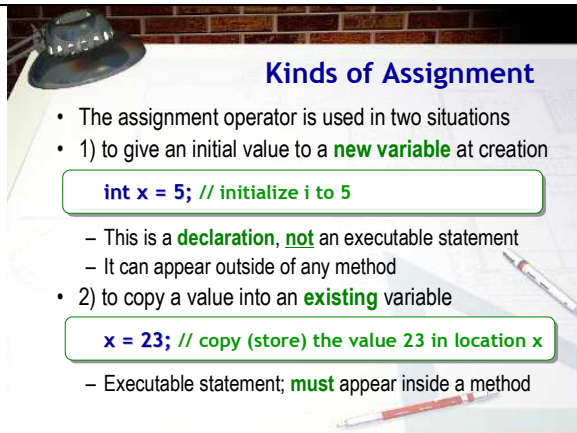
A variable, you'll recall, is a memory location that contains a value. So, the most common thing you'll do with your variables is to change the value that they contain.

You do this through assignment.

The assignment operator is a binary operator whose operation is to:

Copy the value on the right into the variable on the left, and return the result copied as its value

When you apply the assignment operator, the fact that the variable on the left is changed is a side effect of the assignment (although for most of us, it's really the main effect). Because of this side-effect, the operand on the left side of the assignment operator must be a variable; it cannot be a constant or a method invocation.



Kinds of Assignment

- The assignment operator is used in two situations
- 1) to give an initial value to a **new variable** at creation


```
int x = 5; // initialize i to 5
```

 - This is a **declaration**, not an executable statement
 - It can appear outside of any method
- 2) to copy a value into an **existing** variable


```
x = 23; // copy (store) the value 23 in location x
```

 - Executable statement; **must** appear inside a method

Slide 3

Kinds of Assignment

Duration: 00:00:44

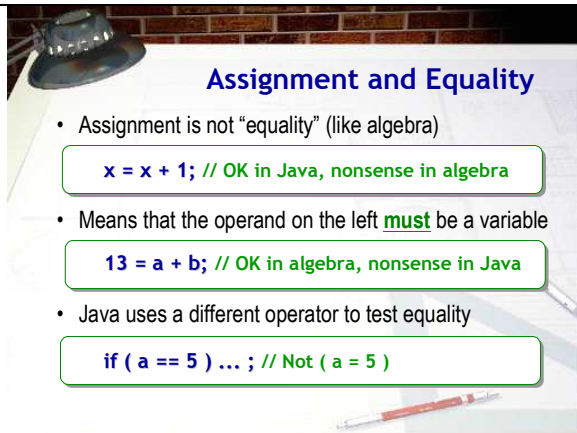
Advance mode: Auto

Notes:

We can use the assignment operator in two different situations.

When you first create a variable, you can give it an initial value using the assignment operator. When you use the assignment operator as part of a variable declaration like this, it is not considered an executable statement, which means that it can appear outside of any method. You used such initializing assignments to provide an initial value for your instance variables.

You can give an existing variable a new value by using an executable assignment. Executable assignments must be placed inside a method, just like all other executable statements.



Assignment and Equality

- Assignment is not "equality" (like algebra)


```
x = x + 1; // OK in Java, nonsense in algebra
```
- Means that the operand on the left **must** be a variable


```
13 = a + b; // OK in algebra, nonsense in Java
```
- Java uses a different operator to test equality


```
if ( a == 5 ) ... ; // Not ( a = 5 )
```

Slide 4

Assignment and Equality

Duration: 00:01:15

Advance mode: Auto

Notes:

Finally, remember that assignment is not equality. You might find this confusing because in math the "equals" sign used in an equation does not mean "copy"; instead it means that the value on the left and the value on the right are identical.

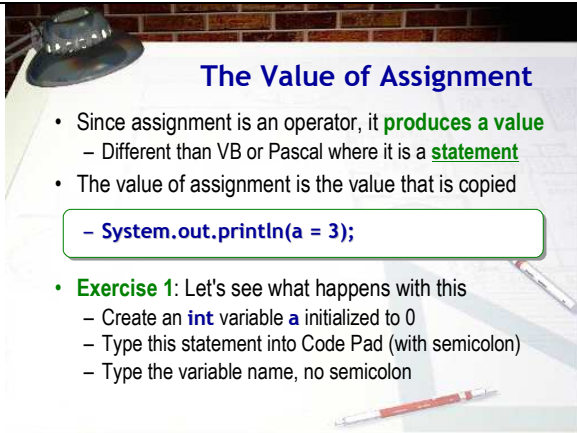
Here's an example. In Algebra, you could never say

$$x = x + 1$$

because for every value of x , $x + 1$ is never equal to x . In Java, however, this statement simply means to add x and 1 together, and then store the result of the operation back into the variable x .

This works both ways. In Algebra, $13 = a + b$ is legal because there are several values for a and b that equal 13. In Java, though, it is illegal (and nonsense), because 13 is not a variable that can be changed. When you use the assignment statement, remember, the right-hand side must be a variable.

Java does have an operator that checks for equality, though, used in if statements and loops. Instead of a single equals sign, Java's equality operator uses a pair of equal signs like this.



The Value of Assignment

- Since assignment is an operator, it **produces a value**
 - Different than VB or Pascal where it is a **statement**
- The value of assignment is the value that is copied

– `System.out.println(a = 3);`
- Exercise 1:** Let's see what happens with this
 - Create an **int** variable **a** initialized to 0
 - Type this statement into Code Pad (with semicolon)
 - Type the variable name, no semicolon

Slide 5 

The Value of Assignment

Duration: 00:01:32
Advance mode: Auto

Notes:

Now, let's look at something new about the assignment operator. Since assignment is an operator, it produces a value, unlike languages like Pascal or Visual Basic where assignment is purely a statement that carries out the copy operation.

The value produced by using the assignment operator is the value that was copied. Here's an example. What do you suppose will happen if you write this:

```
System.out.println(a = 3); ?
```

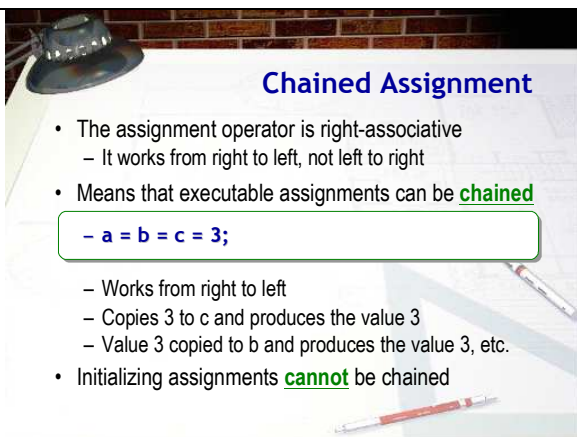
Let's see.

Create a new section in your IC exercise document and then open up BlueJ. In the Code Pad window:

- Create an int variable a initialized to 0
- Type the line shown here. Don't forget the semicolon.
- On the next line, simply type the variable name, a, with no semicolon, and hit Enter

Shoot me one screen-shot of your Code Pad window and one of the BlueJ Console with your output.

When you used `System.out.println()` to display the expression, Java first performed the assignment, storing 3 in the variable a (that's the side-effect part). It then returned the 3 as the expression value, and that's what `System.out.println()` used as it's parameter. When you display the value of a on the last line, you can see that it has been modified.



Chained Assignment

- The assignment operator is right-associative
 - It works from right to left, not left to right
- Means that executable assignments can be **chained**

– `a = b = c = 3;`

 - Works from right to left
 - Copies 3 to c and produces the value 3
 - Value 3 copied to b and produces the value 3, etc.
- Initializing assignments **cannot** be chained

Slide 6 

Chained Assignment

Duration: 00:01:14
Advance mode: Auto

Notes:

Unlike the other binary operators, the assignment operator is right associative; it works from right to left, rather than left to right. Because of this, we can "chain" assignment statements together like this:

```
a = b = c = 3;
```

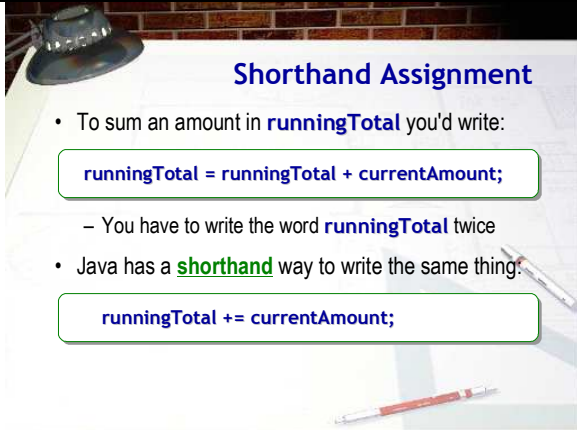
This expression first applies the assignment operation to the variable c, (the right-most part of the expression), copying the value 10 into the variable. The expression `c = 3` produces a value, (3), which is used as the right-hand operand in the next assignment, where 3 is copied into b. In the same way, assigning 3 to b produces a value (3 again), and that value is used as the right-hand operand for the final assignment to a.

There is one potential pitfall you may encounter when doing this: it does not work when defining a new variable, only on variables that already exist. That means you cannot write:

```
int a = b = c = 3; // Not OK
```

Instead, to get the desired effect, you must write:

```
int a = 3, b = 3, c = 3; // Initialization
```



Shorthand Assignment

- To sum an amount in **runningTotal** you'd write:

```
runningTotal = runningTotal + currentAmount;
```

 - You have to write the word **runningTotal** twice
- Java has a **shorthand** way to write the same thing:

```
runningTotal += currentAmount;
```

Slide 7

Shorthand Assignment

Duration: 00:00:46

Advance mode: Auto

Notes:

Many times, when you perform an operation on a variable, you store the result of that operation right back in the same variable. If you are summing up the total in a checkbook, for instance, this would be a common expression:

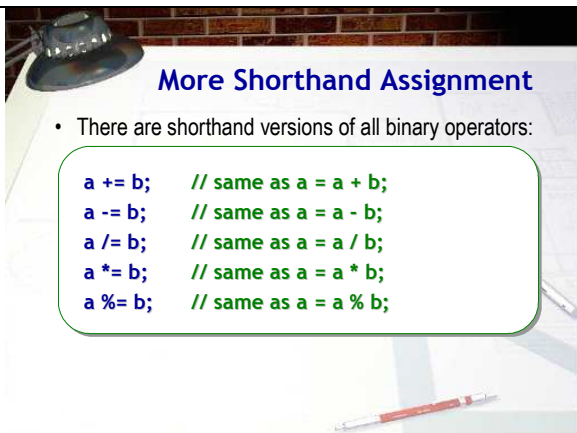
```
runningTotal = runningTotal + currentAmount.
```

What we really want to do is just add `currentAmount` to `runningTotal`, and leave the result in the `runningTotal` variable. The shorthand-assignment operators give us a concise way to do that.

Using these operators we could write the previous expression as:

```
runningTotal += currentAmount;
```

There are shorthand assignment operators corresponding to most of the basic numeric operators.



More Shorthand Assignment

- There are shorthand versions of all binary operators:

```
a += b;    // same as a = a + b;  
a -= b;    // same as a = a - b;  
a /= b;    // same as a = a / b;  
a *= b;    // same as a = a * b;  
a %= b;    // same as a = a % b;
```

Slide 8

More Shorthand Assignment

Duration: 00:00:45

Advance mode: Auto

Notes:

Let's look at each of these. You've already seen the first one.

- `a += b` is the same as `a = a + b`;
- `a -= b` is the same as `a = a - b`
- `a /= b` means `a = a / b`
- `a *= b` works like `a = a * b`
- and `a %= b` is the same as `a = a % b`

These all look pretty straightforward, but you might need to think about them a bit if the "b" part is something other than a simple variable. Let's try some examples.

Express Yourself

- **Exercise 2:** Write equivalent "shorthand" if possible:

```
- int x = 2, y = 3, z = 4, sum = 5, num = 6;  
  x = 2 * x;  
  x = x + y - 2;  
  sum = sum + num;  
  z = z * x + 2 * z;  
  y = y / (x + 5);
```

```
> int x = 2, y = 3, z = 4, sum = 5, num = 6;  
> x = 2 * x;  
4 (int)  
> x = x + y - 2  
5 (int)  
> sum = sum + num  
11 (int)  
> z = z * x + 2 * z  
28 (int)  
> y = y / (x + 5)  
0 (int)  
> y = 27
```

Slide 9

Express Yourself

Duration: 00:00:41

Advance mode: Auto

Notes:

For Exercise 2, I want you to explore the different short-hand assignment operators and how they work with different kinds of expressions. Open up Code Pad and type in the initialization line that creates the five different variables. Then, replace each of the following expressions with a shorthand assignment, if that's possible.

To check to see if you've done it right, I've typed the original expressions into Code Pad so you can see what values you should get for each expression.

Shoot me a screen-shot of your Code Pad session.

Increment and Decrement

- Adding or subtracting 1 from a variable is common
 - Called **increment** (add) and **decrement** (subtract)
- Can do this with the addition, subtraction, assignment

```
int a = 5;  
a = a + 1; // increment  
a = a - 1; // decrement
```

- Creates a temporary value
- Doesn't take advantage of special CPU instructions

Slide 10

Increment and Decrement

Duration: 00:01:00

Advance mode: Auto

Notes:

A common operation in most computer programs is to add or subtract one from a variable and then store the result back into the same variable. This is called incrementing or, (when subtracting), decrementing the variable.

This is easily done using the operators you've met already (addition, subtraction, and assignment) like this:

```
int a = 5;  
a = a + 1; // Increment  
a = a - 1; // Decrement
```

One problem with this solution is that the expression `a + 1` produces a temporary value (like all expressions) that must be stored before it is finally copied back into the variable `a` by the assignment operator.

Since most computer hardware contains special instructions to add one to, (or subtract one from), a variable "in place" (that is, without creating a temporary variable), Java provides operators that use these more efficient instructions.

Up and Down

- Increment and decrement operators
 - Unary operators (`++`, `--`) that add or subtract one
 - Can **only** be applied to a variable
- You may use the operator before or after the variable

```
int a = 5;  
a++; // a is now 6  
++a; // a is now 7
```

- Both have **exactly the same side-effect** on variable
- Variable is **always** incremented or decremented by 1

Slide 11

Notes:

The increment (`++`) and decrement (`--`) operators are unary operators that can only be applied to a variable; you can't use them with literals, constants, or method calls.

You can use the increment or decrement operators immediately before or immediately after a variable, like this:

```
int a = 5;  
a++;  
++a;
```

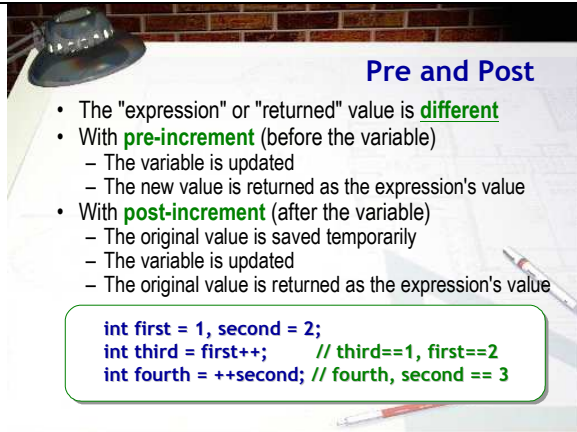
In both cases, the value that was previously stored in `a` is incremented by 1. Since the increment operator modifies its operand, like the assignment operator, this behavior is called

Up and Down

Duration: 00:00:47

Advance mode: Auto

a side effect. In addition to this side effect, the increment operator also produces a value, which has some interesting properties, as you'll see next.



Pre and Post

- The "expression" or "returned" value is **different**
- With **pre-increment** (before the variable)
 - The variable is updated
 - The new value is returned as the expression's value
- With **post-increment** (after the variable)
 - The original value is saved temporarily
 - The variable is updated
 - The original value is returned as the expression's value

```
int first = 1, second = 2;  
int third = first++; // third==1, first==2  
int fourth = ++second; // fourth, second == 3
```

Slide 12

Pre and Post

Duration: 00:01:53

Advance mode: Auto

Notes:

As I just mentioned, it doesn't make any difference whether you place the operator in front of the variable or after the variable as far as the side-effect value goes. The number is always incremented or decremented by 1. It does make a difference when it comes to the expression or returned value, though.

When the operator is placed before the variable, (called a pre-increment or decrement, depending upon the operator) then:

- the variable is updated to its new value
- the new value is returned as the expression value

When the operator is placed after the variable, (called a post-increment or decrement) then:

- the original value stored in the variable is saved temporarily
- the variable is updated to its new value
- the original saved value is returned as the expression value

As far as the side effect is concerned it doesn't make a bit of difference whether the operator is placed in front of, or after, a variable. In both cases, the variable is left holding a value one greater (or less) than it was before. The expression value produced by an increment or decrement, however, depends on whether it is a post or pre expression.

This can be confusing, so here are some examples:

- Line 2 uses post-increment, so the value stored in first (1) is first saved, first is then updated to 2, and finally the saved value (1) is returned and stored in the variable third.
- Line 3 uses pre-increment so the value stored in second (2) is updated to 3 and that new value is returned to be stored in the variable fourth.

Express Yourself

- **Exercise 3:** What value does each variable contain after each statement is executed? (U if not defined)
- `int a = 5, b = 6, c;`
`a = b++ + 3;`
`c = 2 * a + ++b;`
`b = 2 * ++c - a++;`

Slide 13

Express Yourself

Duration: 00:00:27

Advance mode: Auto

Notes:

OK, it's your turn to "Be the Computer" again. Create a column for each of these variables and trace them in your exercise document. (Remember that tracing simply means mentally executing the line of code and then entering the state or value of each variable in the appropriate column.) You don't need to supply a screen-shot for this exercise.

Mixed-type Expressions

- What happens when Java evaluates this expression?
`x = 5.3 * 2;`
 - 5.3 is a **double**, but 2 is an **int**
- With these expressions, Java **promotes** the 2
 - Creates temporary value, 2.0, to use in the expression
 - Called a promotion, because no data is lost
 - The expression result will be a **double**, not an **int**
 - Most precise type used in calculation

Slide 14

Mixed-type Expressions

Duration: 00:02:29

Advance mode: Auto

Notes:

Not all expressions involve integers. You can also have expressions using floating-point numbers, characters, and Strings. You can even have expressions that involve all of the above.

Remember, every expression produces a value, and each value produced by an expression has a particular type. When you add or subtract two integers, for instance, the result of that expression is an integer. When you add or subtract two floating-point numbers, the result is a floating point number. If you perform an operation with a character, the result is a character, and with a String, the result is a String.

Simple, but . . .

What happens when you write this?

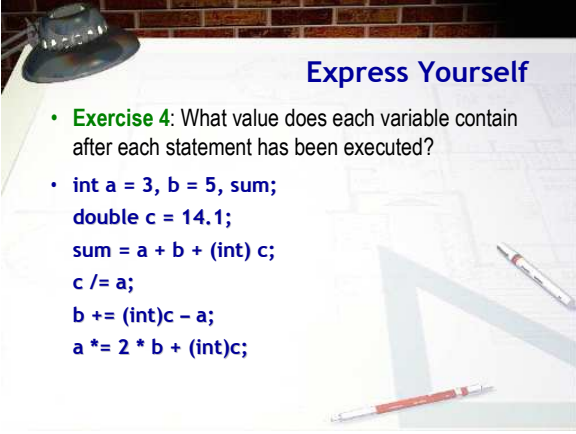
```
a = 5.3 * 2;
```

Whoa! Now we've got a problem. You know that the literal 5.3 is a double. You also know that the literal 2 is an int. What you don't know is the result of multiplying 5.3 * 2, or what type the variable a is. Let's look at both those questions.

Not all numeric types are created equal. A short can hold more information (a greater range of values) than a byte, and an int can hold more information than a short. Likewise, a double can hold more information than a float--it is more precise.

When Java encounters an expression that uses different types of operands, it first determines the most precise of the operands using this data hierarchy. Then, it creates temporary, unnamed variables of the most precise type, initializing those temporary variables using the less-precise values. This process is called promotion, or, a widening conversion.

Let's look at our example again. Of our two values, 5.3 and 2, 5.3 has greater precision--it is higher in the data hierarchy. To perform the calculation, Java creates an unnamed temporary

	double variable that contains 2.0, to take the place of the int value 2 during the calculation. The int value 2 is not changed in any way by this. Finally, the multiplication is performed using the double values 2.0 and 5.3 and the result is the double value 10.6.
 Express Yourself • Exercise 4: What value does each variable contain after each statement has been executed? • <code>int a = 3, b = 5, sum;</code> <code>double c = 14.1;</code> <code>sum = a + b + (int) c;</code> <code>c /= a;</code> <code>b += (int)c - a;</code> <code>a *= 2 * b + (int)c;</code> Slide 15 🎧 Express Yourself Duration: 00:00:27 Advance mode: Auto	Notes: Let's finish up with one last exercise. Create a table for each of the four variables a, b, c and d. Then, mentally execute each line of code and place the appropriate values in each column. When you're finished, use Code Pad to double check your original answers and post a screen-shot below your table.